

Contents

01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide	2
Goja schema map protobuf namespace analysis and implementation guide	2
Executive summary	2
Problem statement and scope	2
Terms and mental model	3
Current-state architecture	5
Reproduction evidence	7
Gap analysis	8
Proposed design	9
Decision records	11
Are there other lossy map[string]any problems?	12
Should go-go-goja be upgraded?	13
Implementation guide for a new intern	13
Test strategy	16
Risks and review notes	16
References	17
01-investigation-diary	19
Diary	19
Goal	19
Step 1: Initial schema map investigation and reverted speculative patch	19
Step 2: Ticket creation, reproducible scripts, and intern-ready design guide	21
Step 3: Retroactively preserve investigation scripts	23
Step 4: Validation and reMarkable delivery	24
Step 5: Tighten the design to remove object descriptor fallback	26
Step 6: Implement strict bulk schema registration	27
tasks	31
Tasks	31
TODO	31
changelog	32
Changelog	32
2026-06-15	32
2026-06-15	32
2026-06-15	32
2026-06-16	32
2026-06-16	32
2026-06-16	33

01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide

Goja schema map protobuf namespace analysis and implementation guide

Executive summary

The sessionstream JavaScript module advertises a bulk schema API where callers can write:

```
const ss = require("sessionstream");
const pb = require("sessionstream.examples.chatdemo.v1");

const schemas = ss.schemas({
  commands: {
    ChatStartInference: pb.StartInferenceCommand,
  },
});
```

That API is not currently reliable. The current implementation decodes the entire JavaScript object through `goja.Runtime.ExportTo` into a Go struct containing `map[string]any` fields. This creates two distinct bugs:

1. The public TypeScript contract uses lower-case section keys (`commands`, `events`, `uiEvents`, `entities`), but the observed `ExportTo` behavior did not populate the Go struct fields from those lower-case keys in the reproduction script. The resulting registry is empty, and later `hub.submit(...)` fails with unknown command "ChatStartInference".
2. If the caller accidentally uses capitalized section names (`Commands`) so that `ExportTo` does populate the Go struct, each generated protobuf namespace object is reduced to a plain `map[string]any`. That strips the hidden `protogoja` schema/prototype token. The exported map keeps public properties such as `typeName`, `builder`, `from`, `is`, and `clone`, but `api_schemas.go` currently only recognizes a public type property in map specs.

The proper local fix is to stop using `ExportTo` for schema leaves. `ss.schemas(input)` should walk the original `goja.Value` object, read the four known sections directly, iterate each section object's own keys, and pass each original leaf `goja.Value` to the same resolver used by the fluent `schemas.registerCommand(...)` methods. That preserves hidden generated-protobuf metadata and aligns the bulk API with the fluent API.

The tightened API should support exactly the two forms already advertised by TypeScript: generated `MessageNamespace` objects and protobuf full-name strings. It should not support plain object descriptors such as `{ type: "..."` } or `{ typeName: "..."` }. Those descriptors are not part of the public contract, are easier to spoof, and only exist in the current code because the lossy `ExportTo` path created plain Go maps.

Changing `go-go-goja` itself is optional, not required for the sessionstream fix. The best upstream improvement would be documentation about preserving original `goja.Value` objects until `protogoja` hidden-reference helpers have run. `MessagePrototypeFromValue` should continue to mean hidden-token extraction, not public `typeName` lookup.

Problem statement and scope

User-visible problem

The advertised API says this should work:

```
export interface SchemaMap {
  commands?: Record<string, string | MessageNamespace>;
  events?: Record<string, string | MessageNamespace>;
  uiEvents?: Record<string, string | MessageNamespace>;
  entities?: Record<string, string | MessageNamespace>;
}
```

```
export function schemas(input?: SchemaMap): Schemas;
```

Source: pkg/js/modules/sessionstream/typescript.go:8-24.

In JavaScript, MessageNamespace means values such as pb.StartInferenceCommand. These are generated by protoc-gen-goja-builder and represent protobuf message types. They are schema tokens, not payloads.

Implementation scope

This guide covers:

- sessionstream/pkg/js/modules/sessionstream/api_schemas.go, where bulk schema input is decoded and schema names are registered.
- sessionstream/pkg/js/modules/sessionstream/module.go, where protobuf full names are resolved and generated namespace hidden tokens are consumed.
- sessionstream/pkg/js/modules/sessionstream/codec.go, where payload values are decoded separately from schema tokens.
- go-go-goja/pkg/protogoja, which defines hidden references for generated protobuf namespace objects and built protobuf message objects.
- go-go-goja/cmd/protoc-gen-goja-builder, which generates the JavaScript namespace API and TypeScript declarations.

This guide intentionally does not implement code. It records the current evidence, design choice, and implementation plan for a follow-up patch.

Terms and mental model

Schema registry

sessionstream.SchemaRegistry stores prototype protobuf messages keyed by logical names such as ChatStartInference or ChatMessage. It clones registered messages on input and lookup.

Relevant implementation:

- pkg/sessionstream/schema.go:11-18: registry maps for commands, events, UI events, and timeline entities.
- pkg/sessionstream/schema.go:29-43: public registration methods.
- pkg/sessionstream/schema.go:85-101: register validates names and clones prototypes.
- pkg/sessionstream/schema.go:104-127: lookup and instantiation clone or create fresh messages.

Generated protobuf namespace object

A generated namespace object is the JavaScript value exported as pb.StartInferenceCommand. It represents a protobuf type. It is not a concrete protobuf message payload.

Generated TypeScript shape:

```

export interface MessageNamespace<TMessage, TBuilder> {
  readonly typeName: string;
  builder(): TBuilder;
  from(value: TMessage): TMessage;
  is(value: unknown): value is TMessage;
  clone(value: TMessage): TMessage;
}

```

Evidence:

- examples/chatdemo/gen/sessionstream/examples/chatdemo/v1/chat_goja.pb.go:45-57: generated TypeScript namespace interface.
- examples/chatdemo/gen/sessionstream/examples/chatdemo/v1/chat_goja.pb.go:370-390: NewStartInferenceCommandGojaNamespace constructs a namespace object, attaches a hidden prototype, and sets public typeName and builder.
- go-go-goja/cmd/protoc-gen-goja-builder/internal/generator/generator.go:420-475: generator template for namespace construction.

Built protobuf message value

A built message is a JavaScript value returned by:

```
pb.StartInferenceCommand.builder().prompt("hello").build()
```

This value carries a hidden `protogoja.MessageRef`. Go can recover the concrete protobuf payload with `protogoja.MessageFromValue(value)`.

Evidence:

- go-go-goja/pkg/protogoja/ref.go:57-99: `ToValue` wraps a protobuf message in a JavaScript object with `typeName`, `toJSON`, `clone`, and `equals`.
- go-go-goja/pkg/protogoja/ref.go:102-128: `MessageFromValue` and `MessageRefFromValue` recover the hidden message reference.
- go-go-goja/cmd/protoc-gen-goja-builder/internal/generator/generator.go:497-503: generated `build()` returns `protogoja.ToValue(vm, builder.Build())`.

Hidden prototype token

A generated namespace object carries a hidden Go-owned prototype token. Go can recover it with `protogoja.MessagePrototypeFromValue(value)`. This is the correct schema-token path because it does not trust public JavaScript properties.

Evidence:

- go-go-goja/pkg/protogoja/prototype.go:11: hidden property key `__go_go_goja_proto_message_pro`
- go-go-goja/pkg/protogoja/prototype.go:50-73: `AttachMessagePrototype` stores the token as non-enumerable.
- go-go-goja/pkg/protogoja/prototype.go:76-91: `MessagePrototypeFromValue` reads the hidden token.
- sessionstream/pkg/js/modules/sessionstream/module.go:201-208: `moduleRuntime.prototypeFromValue` converts a generated namespace token to a fresh prototype message.

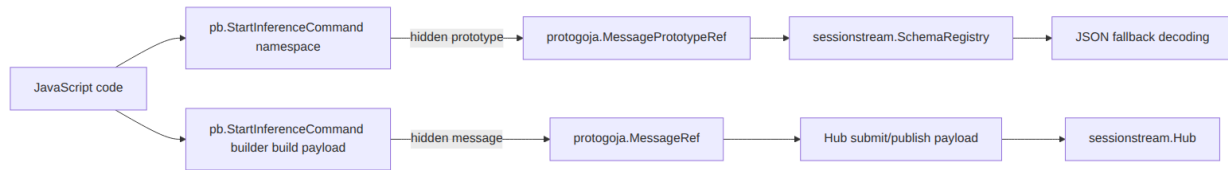


Figure 1: mermaid diagram 1

Current-state architecture

High-level object flow

There are two different hidden-reference paths:

- Schema namespaces use MessagePrototypeRef.
- Built message payloads use MessageRef.

Conflating the two leads to incorrect fixes. `ss.schemas(...)` needs a schema/prototype token. `hub.submit(...)` needs a payload message.

Current bulk schema path

Current schemasBuilder is implemented as:

```

type schemaInput struct {
    Commands map[string]any `json:"commands"`
    Events    map[string]any `json:"events"`
    UIEvents  map[string]any `json:"uiEvents"`
    Entities  map[string]any `json:"entities"`
}

input := schemaInput{}
if arg := call.Argument(0); !goja.IsUndefined(arg) && !goja.IsNull(arg) {
    if err := m.vm.ExportTo(arg, &input); err != nil { ... }
}
for name, spec := range input.Commands {
    msg := m.mustResolveSchemaSpec(spec)
    registry.RegisterCommand(name, msg)
}

```

Evidence: `pkg/js/modules/sessionstream/api_schemas.go:11-53`.

The resolver for exported specs is:

```

func (m *moduleRuntime) mustResolveSchemaSpec(spec any) proto.Message {
    switch v := spec.(type) {
    case string:
        return resolvePrototype(v)
    case map[string]any:
        if full, ok := v["type"].(string); ok {
            return resolvePrototype(full)
        }
    }
    value := m.vm.ToValue(spec)
    if msg, _, ok := m.prototypeFromValue(value); ok {

```

```

        return msg
    }
    panic(TypeError)
}

```

Evidence: pkg/js/modules/sessionstream/api_schemas.go:56-78.

This function tries to reconstruct a Goja value after ExportTo has already copied the JavaScript value to plain Go data. That is too late for hidden metadata: non-enumerable Go-owned references do not survive as ordinary Go maps.

Current fluent schema path

The fluent methods do not use ExportTo:

```

m.mustSet(obj, "registerCommand", func(name string, schema goja.Value) goja.Value {
    msg := m.mustResolveSchemaValue(schema)
    registry.RegisterCommand(name, msg)
    return obj
})

```

Evidence: pkg/js/modules/sessionstream/api_schemas.go:80-112.

The schema value resolver first checks the hidden prototype token:

```

func (m *moduleRuntime) mustResolveSchemaValue(value goja.Value) proto.Message {
    if msg, _, ok := m.prototypeFromValue(value); ok {
        return msg
    }
    if value != nil && !goja.IsUndefined(value) && !goja.IsNull(value) {
        if full := value.String(); full != "" && full != "[object Object]" {
            return resolvePrototype(full)
        }
    }
    panic(TypeError)
}

```

Evidence: pkg/js/modules/sessionstream/api_schemas.go:114-128.

This is why the existing test passes:

```

const schemas = ss.schemas();
schemas.registerCommand("ChatStartInference", pb.StartInferenceCommand);

```

Evidence: pkg/js/modules/sessionstream/module_test.go:46-61.

Current payload decoding path

Payload handling is separate and mostly correct. jsValueToProto first checks for a generated built message:

```

if msg, ok := protogoja.MessageFromValue(value); ok {
    validateMessageType(registry, kind, name, msg)
    return proto.Clone(msg), nil
}

```

Only if that fails does it export the JavaScript value to JSON-compatible data and decode with protojson using the registered schema prototype.

Evidence: pkg/js/modules/sessionstream/codec.go:42-68.

This fallback is intentionally lossy because it is a JSON payload boundary. It is not a schema-token boundary. That difference is central to the design.

Reproduction evidence

All scripts used to investigate this ticket are stored in the ticket workspace under:

`ttp/2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-`

Script 01: lossy boundary inventory

`01-search-lossy-js-boundaries.sh` searches `sessionstream` JavaScript-facing code for `ExportTo`, `map[string]any`, `map[string]interface{}`, and `.Export()`.

The relevant output is in `sources/01-lossy-js-boundaries.txt`. The only schema-token `ExportTo` boundary found is:

```
pkg/js/modules/sessionstream/api_schemas.go:12: Commands map[string]any `json:"commands"`
pkg/js/modules/sessionstream/api_schemas.go:25: if err := m.vm.ExportTo(arg, &input); err
pkg/js/modules/sessionstream/api_schemas.go:64: case map[string]any:
```

Other `map[string]any` usages are JSON output or event/snapshot/fanout shapes, not schema-token inputs.

Script 02: failure reproduction

`02-reproduce-bulk-schema-failure.sh` creates a temporary Go test and removes it after running. It proves both failure modes.

Observed output in `sources/02-reproduce-bulk-schema-failure.txt`:

```
--- FAIL: TestTmpBulkSchemaLowercaseNamespaceFails
Error: GoError: unknown command "ChatStartInference"
```

```
--- FAIL: TestTmpBulkSchemaCapitalizedNamespaceFailsAfterExportToMap
Error: TypeError: schema values must be protobuf full-name strings or generated message na
Interpretation:
```

- Lower-case commands matches TypeScript but did not populate `schemaInput.Commands` in the reproduction. The registry remained empty.
- Capitalized Commands populated the map, but the generated namespace object became a plain map and lost the hidden prototype token.

Script 03: ExportTo inspection

`03-inspect-goja-export-loss.sh` runs a small temporary Go program. It directly inspects the exported `schemaInput` values.

Observed output in `sources/03-inspect-goja-export-loss.txt`:

```
CASE 1: lower-case SchemaMap key used by TypeScript contract
ExportTo error: <nil>
commands nil: true
```

```
CASE 2: capitalized Go struct field name
ExportTo error: <nil>
commands nil: false
```

```
command "ChatStartInference" exported as map[string]interface {}
  keys: [builder clone from is typeName]
  typeName: sessionstream.examples.chatdemo.v1.StartInferenceCommand (string)
  type: <nil> (<nil>)
```

This confirms that the `map[string]any` branch in `mustResolveSchemaSpec` is seeing a plain exported object, not the original generated namespace.

Gap analysis

Gap 1: bulk API does not match TypeScript contract

The TypeScript declaration promises lower-case keys. The implementation uses a Go struct and `ExportTo`, which did not populate from lower-case keys during reproduction.

Impact:

- Users following TypeScript examples can create an empty schema registry without an immediate error.
- The error appears later at `hub.submit`, which makes the root cause hard to discover.

Gap 2: schema tokens cross a lossy conversion boundary

Generated protobuf namespace objects carry hidden non-enumerable Go references. `ExportTo` converts JavaScript objects into public Go values. That is incompatible with schema-token semantics.

Impact:

- Generated namespace values that work in `schemas.registerCommand(...)` fail in `schemas({ ... })`.
- The same public value has inconsistent behavior depending on which API form is used.

Gap 3: undocumented object descriptors are accidental API surface

The bulk `map[string]any` branch recognizes `{ type: "full.name" }`, but the TypeScript `SchemaMap` contract only advertises `string | MessageNamespace`. The object descriptor form is an implementation artifact of the lossy `ExportTo` path, not a clear public API.

Impact:

- Fixing the current bug by expanding the map branch to also accept `{ typeName: "..." }` would promote a third schema-value form that callers do not need.
- Keeping descriptors would blur the intended distinction between generated namespace tokens and explicit protobuf full-name strings.

Gap 4: no regression test covers the advertised bulk namespace form

There is an existing test for the fluent path, but not for:

```
ss.schemas({ commands: { ChatStartInference: pb.StartInferenceCommand } })
```

Evidence: `pkg/js/modules/sessionstream/module_test.go:46-61` covers fluent `registerCommand`, not bulk input.

Proposed design

Design goal

Make the following forms equivalent:

```
// Form A: fluent registration
const schemasA = ss.schemas()
    .registerCommand("ChatStartInference", pb.StartInferenceCommand);

// Form B: bulk registration with generated namespace
const schemasB = ss.schemas({
    commands: {
        ChatStartInference: pb.StartInferenceCommand,
    },
});

// Form C: bulk registration with string full names
const schemasC = ss.schemas({
    commands: {
        ChatStartInference: "sessionstream.examples.chatdemo.v1.StartInferenceCommand",
    },
});

// Deliberately unsupported: plain object descriptors.
// Use a generated namespace or a string full name instead.
ss.schemas({
    commands: {
        ChatStartInference: { typeName: "sessionstream.examples.chatdemo.v1.StartInferenceComm"
    },
});
```

Core implementation rule

Do not call `ExportTo` on schema input. Keep schema leaves as `goja.Value` until after `mustResolveSchemaValue` has had the chance to inspect hidden `protogoja` metadata.

Proposed Go API sketch

Replace `schemaInput` and `mustResolveSchemaSpec(spec any)` with a direct Goja object walker:

```
type schemaRegistrar func(name string, msg proto.Message) error

func (m *moduleRuntime) schemasBuilder(call goja.FunctionCall) goja.Value {
    registry := ss.NewSchemaRegistry()
    if m.defaultSchemaRegistry != nil && goja.IsUndefined(call.Argument(0)) {
        return m.wrapSchemaRegistry(m.defaultSchemaRegistry)
    }

    if arg := call.Argument(0); !goja.IsUndefined(arg) && !goja.IsNull(arg) {
        input, ok := arg.(*goja.Object)
        if !ok || input == nil {
            panic(m.vm.NewTypeError("schema input must be an object"))
        }
    }
}
```

```

        m.registerSchemaSection("commands", input.Get("commands"), registry.RegisterCommand)
        m.registerSchemaSection("events", input.Get("events"), registry.RegisterEvent)
        m.registerSchemaSection("uiEvents", input.Get("uiEvents"), registry.RegisterUIEvent)
        m.registerSchemaSection("entities", input.Get("entities"), registry.RegisterTimeline)
    }

    return m.wrapSchemaRegistry(registry)
}

func (m *moduleRuntime) registerSchemaSection(section string, value goja.Value, register func(string, goja.Value)) {
    if value == nil || goja.IsUndefined(value) || goja.IsNull(value) {
        return
    }
    obj, ok := value.(*goja.Object)
    if !ok || obj == nil {
        panic(m.vm.NewTypeError("schema %s must be an object", section))
    }
    for _, name := range obj.Keys() {
        msg := m.mustResolveSchemaValue(obj.Get(name))
        if err := register(name, msg); err != nil {
            panic(m.vm.NewGoError(err))
        }
    }
}

```

Then keep mustResolveSchemaValue strict. It should support only:

1. generated namespace hidden prototype tokens;
2. primitive string full names.

```

func (m *moduleRuntime) mustResolveSchemaValue(value goja.Value) proto.Message {
    if msg, _, ok := m.prototypeFromValue(value); ok {
        return msg
    }
    if full, ok := schemaFullNameFromValue(value); ok {
        msg, err := m.resolvePrototype(full)
        if err != nil {
            panic(m.vm.NewGoError(err))
        }
        return msg
    }
    panic(m.vm.NewTypeError(
        "schema must be a generated message namespace or protobuf full name",
    ))
}

```

Notes for the intern implementing this:

- Do not accept arbitrary objects with type or typeName properties. They are not part of the tightened API.
- Do not use value.String() for arbitrary objects. It can produce "[object Object]" and can accidentally accept values that are not primitive strings. Prefer value.Export().(string) for string full names.
- Continue resolving full names through resolvePrototype, which first checks Options.Prototypes and then protoregistry.GlobalTypes.

Flow after the fix

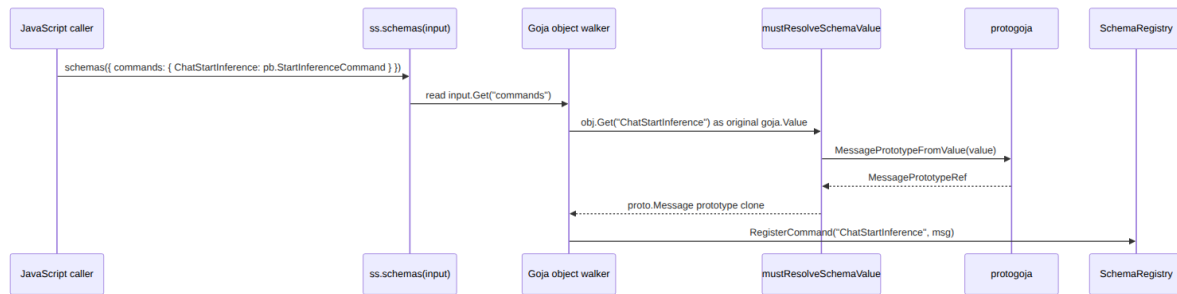


Figure 2: mermaid diagram 2

The important property is that the original `goja.Value` reaches `MessagePrototypeFromValue` unchanged.

Decision records

Decision: Walk Goja schema input directly instead of using ExportTo

- **Context:** Generated protobuf namespace objects carry hidden non-enumerable prototype refs. `ExportTo` copies public data and loses the hidden refs.
- **Options considered:**
 1. Keep `ExportTo` and accept `typeName` in `map[string]any`.
 2. Configure a Goja field-name mapper so lower-case section names populate the struct.
 3. Avoid `ExportTo` and walk the original `goja.Value` object.
- **Decision:** Walk the original Goja object directly.
- **Rationale:** This fixes both the lower-case contract mismatch and hidden-token loss. It also makes bulk registration share the same resolver as fluent registration.
- **Consequences:** Slightly more handwritten object traversal code in `api_schemas.go`, but much clearer schema-token semantics.
- **Status:** proposed.

Decision: Remove object descriptor fallback

- **Context:** Current code partially accepts `{ type: "full.name" }` only because schema leaves are exported to `map[string]any`. The public TypeScript contract is `string | MessageNamespace`, not arbitrary descriptor objects.
- **Options considered:**
 1. Preserve `{ type: "..."` and add `{ typeName: "..."`.
 2. Keep only generated namespaces and primitive string full names.
 3. Remove string support too and require generated namespaces only.
- **Decision:** Keep generated namespaces and primitive string full names; reject object descriptors.
- **Rationale:** This is still unpublished code, so a hard cutover is acceptable. Strings remain useful for config-driven schema maps and for `Options.Prototypes/protoregistry.GlobalTypes`, while object descriptors add a redundant and spoofable third form.
- **Consequences:** Any undocumented caller using `{ type: "..."` must switch to a string full name or generated namespace. The implementation can delete `schemaInput`, `mustResolveSchemaSpec`, and the `map[string]any` branch entirely.
- **Status:** accepted.

Decision: Do not change `protogoja.MessagePrototypeFromValue` to trust `typeName`

- **Context:** It is tempting to fix this in `go-go-goja` by making `MessagePrototypeFromValue` fall back to public `typeName`.
- **Options considered:**
 1. Make `MessagePrototypeFromValue` trust `typeName`.
 2. Add a separate helper for public type-name extraction.
 3. Leave `go-go-goja` unchanged for this fix.
- **Decision:** Do not change `MessagePrototypeFromValue`; optionally add a separate helper/documentation later.
- **Rationale:** `MessagePrototypeFromValue` currently means “this value carries a Go-owned generated prototype token.” Changing it to trust public strings would weaken that semantic boundary and surprise existing consumers.
- **Consequences:** `Sessionstream` owns the fallback policy because `sessionstream` intentionally accepts string schema names. `go-go-goja` remains strict and reusable.
- **Status:** proposed.

Are there other lossy `map[string]any` and `.Export()` sites. The conclusion is that `api_schemas.go` is the only confirmed problematic schema-token boundary.

The inventory script found several `map[string]any` and `.Export()` sites. The conclusion is that `api_schemas.go` is the only confirmed problematic schema-token boundary.

Sites that are intentionally JSON-shaped

These sites build outbound JavaScript objects or event payloads. They do not need to preserve hidden protobuf refs:

- `pkg/js/modules/sessionstream/api_hub.go:143-151`: `snapshotToJS` builds a plain JS snapshot object.
- `pkg/js/modules/sessionstream/api_fanout.go:53-61`: `fanout batch` is a JSON-style event batch.
- `pkg/js/modules/sessionstream/api_view.go:37`: `timeline view` lists plain objects for JavaScript consumption.
- `pkg/js/modules/sessionstream/codec.go:140-160`: UI events and timeline entities are converted to plain output maps.

Payload fallback site

`pkg/js/modules/sessionstream/codec.go:59` calls `value.Export()` only after `protogoja.MessageFromValue(value)` fails. That means built generated protobuf payloads take the non-lossy hidden-reference path first. The fallback is for plain JSON-like payload objects.

Caveat: if a JavaScript object contains nested generated `ProtoMessage` values inside an otherwise plain object, the fallback may not preserve them as protobuf messages. That does not appear to be part of the current API contract. If nested protobuf values become a requirement later, implement a recursive proto-aware conversion helper in `go-go-goja` or `sessionstream` instead of relying on `Export()`.

Hidden sessionstream refs

`pkg/js/modules/sessionstream/module.go:151-168` attaches and reads hidden sessionstream refs for schema registries, hubs, fanouts, and websocket servers. Those refs are

consumed from original `goja.Value` arguments. They are not passed through `ExportTo`, so they are not affected by this bug.

Should go-go-goja be upgraded?

Recommendation

Do not block the sessionstream fix on go-go-goja changes. The sessionstream bug is caused by sessionstream exporting schema-token objects to `map[string]any` before resolving them. A local direct-Goja walker fixes the real boundary.

Useful upstream improvements

A follow-up in go-go-goja can still make future consumers safer:

1. **Documentation update:** Add a warning to `pkg/doc/29-protobuf-builders-user-guide.md` and `cmd/protoc-gen-goja-builder/README.md`: generated namespace and message values must be consumed as original `goja.Values`; `Export`, `ExportTo`, object spread, JSON serialization, or copying only preserves public data such as `typeName`.
2. **Maybe make generated typeName read-only:** `protogoja.ToValue` already uses `defineReadOnly` for built message `typeName` (`ref.go:72`). Generated namespaces currently use `obj.Set("typeName", ...)` in the generator (`generator.go:431`). Making namespace `typeName` read-only would align namespace objects with built messages. It should remain enumerable because it is public API and useful for inspection.
3. **Generator test for export behavior:** Add a test showing that `MessagePrototypeFromValue(namespace)` works on the original namespace, while `ExportTo` produces a plain public object. This documents the boundary rather than hiding it.

Upstream changes to avoid

Avoid changing `protogoja.MessagePrototypeFromValue` so that it constructs a prototype from public `typeName`. That function currently promises hidden-token extraction. Preserving that strong meaning matters for host modules that use it as proof of generated namespace identity.

Implementation guide for a new intern

Phase 0: Read the relevant code

Start with these files in order:

1. `pkg/js/modules/sessionstream/typescript.go`
 - Read the public TypeScript contract for `SchemaMap`, `MessageNamespace`, and `schemas(input)`.
2. `pkg/js/modules/sessionstream/api_schemas.go`
 - Understand the current bulk path and fluent path.
3. `pkg/js/modules/sessionstream/module.go`
 - Read `resolvePrototype` and `prototypeFromValue`.
4. `pkg/js/modules/sessionstream/codec.go`
 - Understand payload decoding so you do not confuse schema tokens with payload messages.
5. `examples/chatdemo/gen/sessionstream/examples/chatdemo/v1/chat_goja.pb.go`

- Read one generated namespace constructor and one builder build() method.
6. /home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/prototype
 - Understand hidden prototype refs.
 7. /home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/ref.go
 - Understand hidden built-message refs.

Phase 1: Add failing tests first

Add tests to pkg/js/modules/sessionstream/module_test.go.

Recommended tests:

```
func TestSchemasBulkRegisterGeneratedPrototypeNamespaces(t *testing.T) {
    vm := goja.New()
    reg := noderequire.NewRegistry()
    Register(reg, Options{})
    require.NoError(t, chatdemov1.RegisterGojaBuilderFileChatProtoModule(reg, ""))
    reg.Enable(vm)

    _, err := vm.RunString(`
const ss = require("sessionstream");
const pb = require("sessionstream.examples.chatdemo.v1");
const schemas = ss.schemas({
    commands: { ChatStartInference: pb.StartInferenceCommand },
    events: { ChatUserMessageAccepted: pb.UserMessageAcceptedEvent },
    uiEvents: { ChatMessageAccepted: pb.ChatMessageUpdate },
    entities: { ChatMessage: pb.ChatMessageEntity },
});
const hub = ss.hub({ schemas });
hub.command("ChatStartInference", (cmd, session, pub) =>
    pub.publish("ChatUserMessageAccepted", pb.UserMessageAcceptedEvent.builder()
        .messageId("m1")
        .role("user")
        .content(cmd.payload.prompt)
        .build())
    );
hub.uiProjection((event) => [{
    name: "ChatMessageAccepted",
    payload: pb.ChatMessageUpdate.builder()
        .messageId(event.payload.messageId)
        .role("user")
        .content(event.payload.content)
        .build(),
}]);
hub.timelineProjection((event) => [{
    kind: "ChatMessage",
    id: event.payload.messageId,
    payload: pb.ChatMessageEntity.builder()
        .messageId(event.payload.messageId)
        .role("user")
        .content(event.payload.content)
        .build(),
}]);`);
}
```

```

        hub.submit("s-1", "ChatStartInference", pb.StartInferenceCommand.builder().prompt(
        `)
    require.NoError(t, err)
}

```

Also add focused resolver tests:

```

// Supported: primitive string full name.
ss.schemas({ commands: { ChatStartInference: "sessionstream.examples.chatdemo.v1.StartInfe

// Unsupported: plain object descriptors.
ss.schemas({ commands: { ChatStartInference: { typeName: "sessionstream.examples.chatdemo.
ss.schemas({ commands: { ChatStartInference: { type: "sessionstream.examples.chatdemo.v1.S

```

Make sure each supported test performs an operation that requires the schema, such as `hub.submit(...)`. Merely constructing `ss.schemas(...)` can hide empty-registry failures. For unsupported object descriptors, assert that `ss.schemas(...)` throws immediately.

Phase 2: Refactor `api_schemas.go`

Perform a small targeted refactor:

1. Remove `fmt` if it is only used for `ExportTo` errors.
2. Delete `schemaInput`.
3. Delete `mustResolveSchemaSpec(spec any)`.
4. Add `schemaRegistrar` and `registerSchemaSection`.
5. Change `schemasBuilder` to use `input.Get("commands")`, `input.Get("events")`, `input.Get("uiEvents")`, and `input.Get("entities")`.
6. Tighten `mustResolveSchemaValue` so it only accepts generated namespaces or primitive string full names.

Be careful with `goja.Value` checks:

```

if value == nil || goja.IsUndefined(value) || goja.IsNull(value) {
    return
}

```

Do not call `ToObject` on undefined or null section values.

Phase 3: Validate behavior

Run:

```

go test ./pkg/js/modules/sessionstream -run 'TestSchemas|TestHubCommandProjectionAndSnapsh
go test ./pkg/js/modules/sessionstream -count=1
go test ./... -count=1

```

If the full test suite is too slow or fails due to unrelated integration dependencies, record the exact failure and at least keep the package-level tests passing.

Phase 4: Optional go-go-goja follow-up

If asked to improve upstream go-go-goja, create a separate ticket/PR. Keep it small:

1. Add docs warning about `ExportTo` lossiness for generated protobuf objects.
2. Consider a helper named clearly around public type names, not hidden prototypes.

3. Add tests under pkg/protogoja or cmd/protoc-gen-goja-builder documenting current hidden-token behavior.

Do not mix this with the sessionstream patch unless the sessionstream patch truly requires a new upstream API.

Test strategy

Required regression tests

Test	Purpose
bulk generated namespace with lower-case commands	Covers the advertised failing API.
bulk string full name with lower-case commands	Ensures string form still works after removing ExportTo.
bulk { typeName } descriptor rejection	Ensures public descriptor objects do not become supported API.
bulk { type } descriptor rejection	Ensures the old map-branch artifact is removed.
fluent registerCommand existing test	Guards against regression in the existing working path.
invalid non-object section	Confirms helpful type errors.

Manual smoke snippet

After implementing, this JavaScript should run without error in a Goja test:

```
const ss = require("sessionstream");
const pb = require("sessionstream.examples.chatdemo.v1");

const schemas = ss.schemas({
  commands: {
    ChatStartInference: pb.StartInferenceCommand,
  },
});

const hub = ss.hub({ schemas });
hub.command("ChatStartInference", () => {});
hub.submit(
  "s-1",
  "ChatStartInference",
  pb.StartInferenceCommand.builder().prompt("hello").build()
);
```

Risks and review notes

Risk: hard cutover removes { type: "..."} }

This is acceptable because the code is still unpublished and the TypeScript contract never advertised object descriptors. Callers should use either a generated namespace object or a primitive string full name.

Risk: iteration order is unspecified

`obj.Keys()` order should not matter because schema registration sections are maps. Duplicate logical names within the same JS object cannot exist as distinct own properties.

Risk: object prototype pollution

Use `obj.Keys()` so only enumerable own keys are registered. Do not walk inherited properties. If stricter behavior is desired later, add tests for `Object.create({ inherited: ... })`.

Risk: default schema registry behavior

Preserve this behavior exactly:

```
if m.defaultSchemaRegistry != nil && goja.IsUndefined(call.Argument(0)) {  
    return m.wrapSchemaRegistry(m.defaultSchemaRegistry)  
}
```

Passing `{}` should create a new empty registry, not return the default registry.

References

Sessionstream files

- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream` — broken bulk schema decoder and fluent schema registration.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream` — module runtime, hidden sessionstream refs, protobuf prototype resolution.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream` — payload decoding and JSON fallback.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream` — public SchemaMap and MessageNamespace declarations.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream` — existing fluent schema test and integration test patterns.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/sessionstream/schema` — schema registry implementation.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/examples/chatdemo/generator` — generated chat demo protobuf Goja namespace and builder code.

Go-go-goja files

- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/prototype` — hidden prototype token API for generated namespaces.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/protogoja/ref.go` — hidden built-message reference API for payload values.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/cmd/protogen-goja-builder/internal/generator/generator.go` — generator template for namespace objects and builders.
- `/home/manuel/workspaces/2026-06-12/goja-sessionstream/go-go-goja/pkg/doc/29-protobuf-builders-user-guide.md` — user-facing generated protobuf builder documentation.

Ticket evidence

- `sources/01-lossy-js-boundaries.txt` — inventory of lossy conversion sites.

- `sources/02-reproduce-bulk-schema-failure.txt` — failing temporary tests.
- `sources/03-inspect-goja-export-loss.txt` — direct `ExportTo` inspection output.
- `scripts/01-search-lossy-js-boundaries.sh` — search script.
- `scripts/02-reproduce-bulk-schema-failure.sh` — failure reproduction script.
- `scripts/03-inspect-goja-export-loss.sh` — export inspection script.

01-investigation-diary

Diary

Goal

This diary records the investigation into lossy Goja schema map handling in sessionstream, including the accidental speculative edit that was reverted, the reproduction scripts that were retained in the ticket workspace, and the analysis/design guide produced for the follow-up implementation.

Step 1: Initial schema map investigation and reverted speculative patch

I first inspected the sessionstream JavaScript schema registration path to understand the review comment around `map[string]any` and generated protobuf namespaces. The key finding was that `ss.schemas({...})` uses `goja.ExportTo` before resolving schema leaves, while the fluent `schemas.registerCommand(...)` path receives the original `goja.Value` and can still recover the hidden `protogoja` prototype token.

I briefly drafted a direct-Goja walker implementation in `pkg/js/modules/sessionstream/api_schemas.go`, then stopped when the user asked to study the system first and create a ticketed analysis/design guide instead. I reverted the speculative code change with `git checkout -- pkg/js/modules/sessionstream/api_schemas.go`, leaving the repository source unchanged before creating the ticket.

Prompt Context

User prompt (verbatim): “Are there other lossy map strings any to be found? we should do the proper fix.”

Assistant interpretation: Search for other lossy `map[string]any` or `ExportTo` boundaries and consider a more principled fix than only adding `typeName` to the existing map branch.

Inferred user intent: Avoid a narrow patch that misses related schema/protobuf object handling problems.

Commit (code): N/A — no code commit; speculative edit was reverted.

What I did

- Read `pkg/js/modules/sessionstream/api_schemas.go`.
- Searched for `ExportTo`, `map[string]any`, and related conversion boundaries.
- Inspected generated protobuf namespace code in `examples/chatdemo/gen/sessionstream/examples/chatdemo`.
- Inspected `go-go-goja/pkg/protogoja/prototype.go` and `ref.go` to understand hidden namespace and payload references.
- Drafted and then reverted a direct-Goja walker patch in `api_schemas.go`.

Why

- The review comment suggested that the current map branch loses hidden generated-protobuf prototype metadata.
- A proper fix needed to distinguish schema namespace objects from built protobuf payload objects.

What worked

- The source inspection confirmed that fluent registration already takes the correct `goja.Value` path.
- The initial patch sketch clarified the shape of the likely implementation: walk the original schema input object directly and reuse `mustResolveSchemaValue`.

What didn't work

- I made a speculative edit before creating the requested analysis ticket. That edit was reverted with:

```
git checkout -- pkg/js/modules/sessionstream/api_schemas.go
```

What I learned

- `map[string]any` is not the root concept; the root problem is crossing a lossy JS-to-Go export boundary before consuming hidden `protogoja` metadata.
- `MessagePrototypeFromValue` and `MessageFromValue` are intentionally separate APIs: schema namespaces use prototype refs; built payload messages use message refs.

What was tricky to build

- The tricky part was recognizing that adding `typeName` to the existing map branch is only a compatibility fallback, not the proper fix. Once `ExportTo` has run, the authoritative hidden token is already gone.
- The other tricky point was not confusing JSON payload fallback in `codec.go` with schema-token handling in `api_schemas.go`; payload fallback is intentionally JSON-shaped after checking `protogoja.MessageFromValue`.

What warrants a second pair of eyes

- Confirm whether any consumers intentionally pass capitalized `Commands/Events` section names today. The public TypeScript contract uses lower-case names.
- Confirm whether object descriptors with `{ type: ... }` should remain supported when refactoring away from `mustResolveSchemaSpec`.

What should be done in the future

- Implement the direct-Goja schema input walker described in the design doc.
- Add failing tests before changing production code.

Code review instructions

- Start at `pkg/js/modules/sessionstream/api_schemas.go` and compare the `bulk_schemas(input)` path with the fluent `registerCommand` path.
- Validate with package-level tests once implementation starts: `go test ./pkg/js/modules/sessionstream -count=1`.

Technical details

- Existing failing area: `pkg/js/modules/sessionstream/api_schemas.go:11-78`.
- Existing working fluent path: `pkg/js/modules/sessionstream/api_schemas.go:80-128`.

- Generated namespace hidden token API: `go-go-goja/pkg/protogoja/prototype.go:50-91`.

Step 2: Ticket creation, reproducible scripts, and intern-ready design guide

After the user asked to pause implementation, I created ticket SS-GOJA-SCHEMA-MAP-001 and moved the investigation into a docmgr workspace. I also retroactively saved the temporary reproduction and inspection programs as durable scripts under the ticket `scripts/` directory so future readers can rerun the exact evidence-gathering steps.

The main deliverable is a detailed design and implementation guide explaining the session-stream schema registry, Goja generated protobuf namespace objects, hidden protogoja references, current failure modes, proposed direct-Goja walker fix, testing strategy, and whether `go-go-goja` itself should change.

Prompt Context

User prompt (verbatim): “hold on, let’s study things first and create a new ticket and an analysis + implemengation guide.

We also want to consider if changing `./go-go-goja/` and its protobuf handling itself makes sense to be upgraded?

Create a new ticket and then:

Create a detailed analysis / design / implementation guide that is for a new intern, explaining all the parts of the system needed to understand what it is, with prose paragraphs and bullet point sand pseudocode and diagrams and api references and file references. It should be very clear and technical. Store in the ticket and the nupload to remarkable.”

Assistant interpretation: Stop implementation, create a docmgr ticket, investigate evidence first, write a comprehensive intern-ready analysis/design/implementation guide, consider upstream `go-go-goja` changes, store everything in the ticket, and upload the result to reMarkable.

Inferred user intent: Produce a durable design artifact before coding so the eventual fix is evidence-based and reviewable.

Commit (code): N/A — documentation and ticket artifacts only.

What I did

- Created ticket SS-GOJA-SCHEMA-MAP-001 with docmgr.
- Added design doc `design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md`.
- Added diary doc reference `01-investigation-diary.md`.
- Added ticket tasks for inventory, reproduction, guide writing, validation, and upload.
- Wrote scripts:
 - `scripts/01-search-lossy-js-boundaries.sh`
 - `scripts/02-reproduce-bulk-schema-failure.sh`
 - `scripts/03-inspect-goja-export-loss.sh`
- Captured script outputs under `sources/`.
- Wrote the intern-ready design guide.

Why

- The task changed from implementation to research/design.
- The reproduction scripts preserve exact commands and prevent future investigators from relying on vague memory of temporary experiments.

What worked

- 02-reproduce-bulk-schema-failure.sh reproduced two failures:
 - lower-case commands led to unknown command "ChatStartInference";
 - capitalized Commands led to TypeError: schema values must be protobuf full-name strings or generated message namespace objects.
- 03-inspect-goja-export-loss.sh showed that lower-case commands did not populate the Go struct in the reproduction, while capitalized Commands exported the namespace as map[string]interface {} with public keys [builder clone from is typeName] and no type.

What didn't work

- The reproduction test intentionally fails under current code. Its output is saved in sources/02-reproduce-bulk-schema-failure.txt and should be interpreted as evidence, not as a validation pass.
- reMarkable upload has not yet been completed at the time of this diary step.

What I learned

- The review comment was directionally correct but understated the lower-case key issue: the advertised TypeScript SchemaMap uses lower-case keys, and the reproduction showed those keys did not populate schemaInput via ExportTo.
- The go-go-goja hidden-token APIs are behaving as designed; sessionstream should avoid destroying the hidden token before asking protogoja to read it.

What was tricky to build

- The reproduction needed to actually submit through the hub. A test that only constructs ss.schemas({...}) can pass even if the registry is empty, because the error appears later when resolving a command payload.
- The temporary inspection program needed to compare lower-case commands with capitalized Commands to expose both distinct failure modes.

What warrants a second pair of eyes

- Review the proposed fallback support for both typeName and type object descriptors.
- Review the recommendation not to change go-go-goja.MessagePrototypeFromValue semantics.
- Review whether namespace typeName should become read-only in generated go-go-goja code as a separate upstream hardening task.

What should be done in the future

- Run docmgr doctor after file relations and changelog updates are complete.
- Upload the ticket bundle to reMarkable after validation.
- Implement the direct-Goja walker in a follow-up code change.

Code review instructions

- Review the design doc first, especially “Proposed design”, “Decision records”, and “Implementation guide for a new intern”.
- Rerun evidence scripts from the ticket root if needed:

```
./scripts/01-search-lossy-js-boundaries.sh
./scripts/02-reproduce-bulk-schema-failure.sh
./scripts/03-inspect-goja-export-loss.sh
```

Technical details

- Ticket path: `ttp/2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling`.
- Design doc: `design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md`.
- Evidence outputs: `sources/01-lossy-js-boundaries.txt`, `sources/02-reproduce-bulk-schema-failure.txt`, `sources/03-inspect-goja-export-loss.txt`.

Step 3: Retroactively preserve investigation scripts

The user explicitly asked that all scripts I write be saved in the ticket `scripts/` folder, including scripts already used. I confirmed the temporary scripts had been converted into tracked ticket scripts and that their outputs were stored in `sources/`.

Prompt Context

User prompt (verbatim): “save all the scripts you write in the `scripts/` folder of the ticket, retroactively too, so we can keep track of your work.”

Assistant interpretation: Ensure every ad-hoc reproduction or inspection script from this investigation exists as a durable script in the ticket workspace.

Inferred user intent: Keep the investigation reproducible and auditable for future continuation or review.

Commit (code): N/A — ticket artifact update only.

What I did

- Saved the lossy boundary search as `scripts/01-search-lossy-js-boundaries.sh`.
- Saved the temporary failing test reproduction as `scripts/02-reproduce-bulk-schema-failure.sh`.
- Saved the temporary `goja.ExportTo` inspection program generator as `scripts/03-inspect-goja-export-loss.sh`.
- Made scripts executable.
- Stored outputs in `sources/`.

Why

- Reproducible scripts make the design guide evidence-backed and easy for an intern to verify.

What worked

- All scripts are now in the ticket workspace and can be rerun from a clean checkout.

What didn't work

- N/A

What I learned

- The ticket workflow should save scripts before or immediately after first execution; doing so retroactively is possible but easier to miss.

What was tricky to build

- The scripts are stored several directories below the repository root, so they use `git -C "$SCRIPT_DIR" rev-parse --show-toplevel` instead of fragile relative `../../..` paths.

What warrants a second pair of eyes

- Confirm the script set is sufficient, or add a fourth script later for validating the final implementation once code changes begin.

What should be done in the future

- Add `04-validate-implemented-fix.sh` after the actual production patch is written.

Code review instructions

- Start with the scripts in numerical order.
- Compare each script output with the corresponding file under `sources/`.

Technical details

- Scripts are under `ttp/2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/scripts/`.

Step 4: Validation and reMarkable delivery

I validated the ticket with `docmgr doctor`, rendered a local PDF bundle, fixed a Mermaid parse issue in the design document, and uploaded the corrected bundle to reMarkable. The final remote listing confirms the bundle exists at the requested date/ticket destination.

The Mermaid issue was caused by a flowchart node label containing `builder().build()` punctuation. I changed that label to plain quoted text, regenerated the PDF successfully, and re-uploaded with `--force` so the reMarkable copy reflects the corrected diagram.

Prompt Context

User prompt (verbatim): (same as Step 2 and Step 3)

Assistant interpretation: Complete ticket validation and upload the generated analysis bundle to reMarkable.

Inferred user intent: Make the design guide available both in the repository ticket workspace and on the reMarkable device.

Commit (code): N/A — documentation and generated PDF only.

What I did

- Ran `docmgr doctor --ticket SS-GOJA-SCHEMA-MAP-001 --stale-after 30`.
- Ran `remarquee status` and `remarquee cloud account --non-interactive`.
- Ran a dry-run bundle upload.
- Ran the real bundle upload.
- Observed Mermaid render warnings, fixed the flowchart label, generated a local PDF with `--pdf-only`, and re-uploaded with `--force`.
- Verified remote listing with `remarquee cloud ls /ai/2026/06/15/SS-GOJA-SCHEMA-MAP-001 --long --non-interactive`.

Why

- The ticket-research workflow requires validation before delivery and a dry-run before upload.
- The first upload succeeded but had Mermaid render warnings, so the bundle needed a corrected re-upload.

What worked

- `docmgr doctor` passed with all checks.
- `remarquee upload bundle --pdf-only` generated `sources/SS-GOJA-SCHEMA-MAP-001 schema map protobuf namespace guide.pdf` without warnings after the Mermaid fix.
- The final remote listing showed:

```
[f] SS-GOJA-SCHEMA-MAP-001 schema map protobuf namespace guide
```

What didn't work

- First real upload produced Mermaid warnings:

```
WARNING: failed to render Mermaid block 1: mmdc failed: Generating single mermaid chart
Error: Parse error on line 3:
...renceCommand.builder().build() payload]
-----^
```

What I learned

- Mermaid flowchart labels that include method-call punctuation should be quoted or simplified for reliable PDF rendering.

What was tricky to build

- The upload command succeeded even while Mermaid rendering warned. I treated the warning as a quality issue, fixed the source document, verified local PDF rendering, and then overwrote the remote copy intentionally with `--force`.

What warrants a second pair of eyes

- Review the rendered PDF if diagram fidelity matters; the corrected flowchart uses a simplified label instead of the exact JavaScript method-call string.

What should be done in the future

- Add a fourth script for validating the implementation after the production code patch is written.

Code review instructions

- Check design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md for the final corrected Mermaid flowchart.
- Check sources/SS-GOJA-SCHEMA-MAP-001 schema map protobuf namespace guide.pdf if reviewing the uploaded PDF contents locally.

Technical details

- reMarkable destination: /ai/2026/06/15/SS-GOJA-SCHEMA-MAP-001.
- Uploaded document name: SS-GOJA-SCHEMA-MAP-001 schema map protobuf namespace guide.

Step 5: Tighten the design to remove object descriptor fallback

The design was updated to reflect the final API choice: bulk schemas should support only generated MessageNamespace objects and primitive protobuf full-name strings. Plain descriptor objects such as { type: "..."} and { typeName: "..."} should be rejected rather than documented or kept for compatibility.

This aligns the implementation plan with the fact that the code is still unpublished. The map[string]any branch is an artifact of ExportTo; removing ExportTo also removes the need for descriptor fallback support.

Prompt Context

User prompt (verbatim): “ok, keep it. Otherwise, we can do a hard cutover and not worry about backwards support, since this is still unpublished code. Update your design documentation to make this tighter API, and add a detailed list of tasks to the ticket.

Then implement them one by one, committing at appropriate intervals, keeping a detailed diary.”

Assistant interpretation: Keep string support, remove object descriptor fallback, update the ticket design and tasks, then implement the fix incrementally with commits and diary updates.

Inferred user intent: Prefer a clean unpublished API over backwards-compatible support for accidental descriptor objects.

Commit (code): pending — documentation/task update before implementation.

What I did

- Added detailed follow-up tasks to the ticket for design tightening, tests, implementation, validation, and diary/changelog updates.
- Updated the design guide to remove { type: ... } / { typeName: ... } support from the proposed implementation.
- Updated decision records to make object descriptor removal an accepted decision.

Why

- The public TypeScript contract is string | MessageNamespace.
- Object descriptors are redundant and spoofable, and were only present because the old implementation exported schema values into maps.

What worked

- The design now states that `schemaInput`, `mustResolveSchemaSpec`, and the `map[string]any` branch should be removed entirely.
- The test plan now includes descriptor rejection tests instead of descriptor acceptance tests.

What didn't work

- N/A

What I learned

- Keeping string support is useful because it exercises `resolvePrototype`, `Options.Prototypes`, and protobuf global type lookup without requiring generated JS namespace modules.
- Keeping object descriptor support is unnecessary for unpublished code and would turn an implementation artifact into public API.

What was tricky to build

- The wording had to distinguish primitive string support from object descriptor fallback. Both contain type names, but only primitive strings are part of the tightened API.

What warrants a second pair of eyes

- Confirm that no downstream unpublished examples rely on `{ type: ... }`. If they do, update them to a string full name or generated namespace.

What should be done in the future

- Implement the stricter resolver and descriptor rejection tests.

Code review instructions

- Review the design guide sections: Executive summary, Gap 3, Proposed design, Decision records, Phase 1/2 implementation guide, and Test strategy.

Technical details

- Updated design doc: <http://2026/06/15/SS-GOJA-SCHEMA-MAP-001--analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md>.

Step 6: Implement strict bulk schema registration

I implemented the stricter schema registration path described by the updated design. Bulk `ss.schemas({...})` now walks the original Goja input object, reads lower-case schema sections directly, and resolves each schema leaf while it is still a `goja.Value`. This preserves generated protobuf namespace hidden prototype tokens and removes the old `ExportTo/map[string]any` schema path entirely.

The resulting API supports exactly generated `MessageNamespace` values and primitive protobuf full-name strings. Plain object descriptors such as `{ type: "..."}` and `{ typeName: "..."}` now fail immediately with the same schema type error as any other unsupported object.

Prompt Context

User prompt (verbatim): (same as Step 5)

Assistant interpretation: Implement the tightened API and keep the diary current while validating each task.

Inferred user intent: Move from design to code with a clean API cutover and strong regression coverage.

Commit (code): 298d0496b4a2661d44ca80f74d5538fe81e28e51 — “Fix bulk schema registration for Goja protobuf namespaces”

What I did

- Added regression tests in `pkg/js/modules/sessionstream/module_test.go`:
 - `TestSchemasBulkRegisterGeneratedPrototypeNamespaces`
 - `TestSchemasBulkRegisterStringFullNames`
 - `TestSchemasRejectPlainObjectDescriptors`
- Verified the new tests fail against the old implementation before changing production code.
- Refactored `pkg/js/modules/sessionstream/api_schemas.go`:
 - removed `schemaInput`;
 - removed `mustResolveSchemaSpec(spec any)`;
 - removed the `map[string]any` branch;
 - removed `goja.Runtime.ExportTo` from `schemasBuilder`;
 - added `registerSchemaSection` over original `goja.Value` objects;
 - tightened `mustResolveSchemaValue` to accept generated namespaces or primitive strings only.
- Ran `gofmt`.

Why

- `ExportTo` stripped hidden generated-protobuf prototype tokens and also failed to populate lower-case TypeScript section keys in the reproduction.
- Removing the exported-map path makes bulk registration behave like fluent registration.

What worked

- The focused tests now pass:

```
go test ./pkg/js/modules/sessionstream -run 'TestSchemas(BulkRegisterGeneratedPrototypeNam
# ok github.com/go-go-golems/sessionstream/pkg/js/modules/sessionstream 0.005s
```

- Package tests now pass:

```
go test ./pkg/js/modules/sessionstream -count=1
# ok github.com/go-go-golems/sessionstream/pkg/js/modules/sessionstream 0.072s
```

- Full repository tests now pass:

```
go test ./... -count=1
# ok github.com/go-go-golems/sessionstream/... (all packages with tests passed)
```

- The pre-commit hook also passed `GOWORK=off go test ./... and golanci-lint run -v` before creating commit 298d0496b4a2661d44ca80f74d5538fe81e28e51.

What didn't work

- The new tests failed as expected before implementation:
--- FAIL: TestSchemasBulkRegisterGeneratedPrototypeNamespaces
Error: GoError: unknown command "ChatStartInference"

--- FAIL: TestSchemasBulkRegisterStringFullNames
Error: GoError: unknown command "ChatStartInference"

--- FAIL: TestSchemasRejectPlainObjectDescriptors/typeName
Error: An error is expected but got nil.

--- FAIL: TestSchemasRejectPlainObjectDescriptors/type
Error: An error is expected but got nil.

What I learned

- Lower-case bulk sections must be read from the original JavaScript object. Relying on Go struct export tags here is both unnecessary and fragile.
- Checking `value.Export().(string)` is stricter than `value.String()` and avoids accidentally treating arbitrary objects as strings.

What was tricky to build

- Promise-returning hub operations can hide asynchronous handler/projection failures if a test only checks `RunString` errors. The bulk generated namespace test now checks that the returned Promise is fulfilled, which exercises command, event, UI event, and timeline entity schemas.
- Descriptor rejection needed to call `ss.schemas(...)` directly after the refactor because schema values are resolved during registration rather than later at submit time.

What warrants a second pair of eyes

- Review `registerSchemaSection` for Goja object semantics, especially whether `obj.Keys()` is the right key set for this API.
- Review the hard cutover from `{ type: "..."} to primitive string full names.`

What should be done in the future

- Consider a small go-go-goja documentation update warning consumers not to call `ExportTo` before `protogoja.MessagePrototypeFromValue / MessageFromValue`.

Code review instructions

- Start with `pkg/js/modules/sessionstream/api_schemas.go` and confirm `ExportTo` is gone from schema registration.
- Then review `pkg/js/modules/sessionstream/module_test.go` for the three new regression tests.
- Validate with:

```
go test ./pkg/js/modules/sessionstream -count=1
go test ./... -count=1
```

Technical details

- Modified code file: `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js`
- Modified test file: `/home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js`

tasks

Tasks

TODO

- ☒ Inventory lossy Goja conversion boundaries in sessionstream and go-go-goja
- ☒ Reproduce the bulk schemas({ ... }) generated namespace failure
- ☒ Write intern-ready analysis, design, and implementation guide
- ☒ Validate ticket docs and upload bundle to reMarkable
- ☒ Tighten design doc to support only string full names and generated MessageNamespace schema values
- ☒ Add regression tests for lower-case bulk schemas with generated namespaces and string full names
- ☒ Add regression test that plain descriptor objects are rejected
- ☒ Refactor api_schemas.go to walk original Goja objects and remove ExportTo/map[string]any schema handling
- ☒ Run focused and package-level validation tests
- ☒ Update diary/changelog/file relations and commit the implementation

changelog

Changelog

2026-06-15

- Initial workspace created

2026-06-15

Created evidence-backed analysis and implementation guide for lossy Goja schema map handling; saved reproduction scripts and outputs for future implementation.

Related Files

- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001-analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md — Primary deliverable
- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001-analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/reference/01-investigation-diary.md — Chronological investigation record

2026-06-15

Validated ticket docs with docmgr doctor and uploaded the analysis bundle to reMarkable at /ai/2026/06/15/SS-GOJA-SCHEMA-MAP-001.

Related Files

- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001-analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/sources/SS-GOJA-SCHEMA-MAP-001 schema map protobuf namespace guide.pdf — Rendered PDF bundle uploaded to reMarkable

2026-06-16

Tightened schema map design to support only generated MessageNamespace values and protobuf full-name strings; object descriptor fallback will be removed.

Related Files

- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/ttmp/2026/06/15/SS-GOJA-SCHEMA-MAP-001-analyze-lossy-goja-schema-map-export-and-protobuf-namespace-handling/design-doc/01-goja-schema-map-protobuf-namespace-analysis-and-implementation-guide.md — Updated API decision and implementation plan

2026-06-16

Implemented strict bulk schema registration by walking original Goja objects, preserving generated namespace prototypes, and rejecting plain descriptor objects.

Related Files

- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream/ — Removed ExportTo/map[string]any schema handling and added direct Goja section traversal
- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream/ — Added bulk namespace

2026-06-16

Committed strict bulk schema registration implementation (298d0496b4a2661d44ca80f74d5538fe81e28e51)

Related Files

- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream/ — Committed direct Goja schema traversal
- /home/manuel/workspaces/2026-06-12/goja-sessionstream/sessionstream/pkg/js/modules/sessionstream/ — Committed regression coverage